



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии (ИУ7)»

ОТЧЕТ

Лабораторная работа №4

по курсу «Проектирование экспертных систем»
на тему: «Процедура резолюции с унификацией»

Студент ИУ7-32М
(Группа)

(Подпись, дата)

К.Э. Ковалец
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

З.Н. Русакова
(И. О. Фамилия)

2024 г.

Содержание

1	Основная часть	3
1.1	Унификация	3
1.2	Резолюция	3
1.3	Алгоритм вывода по методу резолюции	4
1.4	Пример работы программы	4
2	Текст программы	7

1 Основная часть

1.1 Унификация

Унификация — это процесс замены предметных переменных аргумента атома другими предметными переменными или константами или функциональными символами. Подстановка называется унификацией, если атомы совпадают.

Унификация выполняется:

- если термы — константы, то они унифицируются, если совпадают;
- если в первом атоме терм — переменная, во втором — константа, то они унифицируются, и переменная получает значение константы;
- если терм в первом атоме — переменная и во втором — переменная, то они унифицируются и становятся связанными, можно использовать любое имя.

Унифицируются между собой термы, стоящие на одинаковых местах в контрарных атомах.

1.2 Резолюция

Резольвента — это объединение двух дизъюнктов за исключением контрарной пары. Метод формирования резольвенты заключается в следующем:

- сначала для двух атомов формируется список подстановок с помощью процедуры унификации;
- затем эти значения распространяются в атомы первого и второго дизъюнктов, за исключением контрарной пары.

Множество подстановок при унификации атомов и дизъюнктов нужно выполнять последовательно, просматривая каждый раз только одну переменную.

1.3 Алгоритм вывода по методу резолюции

Необходимо создать базу данных, которая будет представлять собой список дизъюнктов.

Алгоритм состоит из следующих шагов:

- принять отрицание заключения (того, что хотим доказать);
- привести все формулы аксиом и отрицание цели к КНФ;
- получить список дизъюнктов;
- если существует пара дизъюнктов, содержащих контрарные атомы, то эти дизъюнкты объединить с удалением контрарной пары;
- если на каком-то шаге получим два дизъюнкта, содержащих одинаковые атомы с разными знаками, то резольвентой будет пустой дизъюнкт или ложь;
- иначе продолжить формирование резольвенты (перебор на множестве).

1.4 Пример работы программы

Результаты работы программы приведены в листинге 1.1.

Листинг 1.1 — Результат работы программы

```
1  База знаний:
2
3  P2(x1:var, y1:var) | P5(z1:var) | ~P6(w1:var)
4  P3(C:const) | ~P4(z2:var) | P1(x2:var, y2:var, z2:var)
5  ~P2(A:const, B:const) | P5(x3:var) | P6(y3:var)
6  P4(x4:var) | ~P3(x4:var)
7
8  -----
9
10 Пара дизъюнктов:
11   1) ~P1(A:const, B:const, C:const)
12   2) P2(x1:var, y1:var) | P5(z1:var) | ~P6(w1:var)
13
14 Отсутствие унификаций
15
16 -----
17
18 Пара дизъюнктов:
```

Продолжение листинга 1.1

```
19      1)  $\sim P1(A:const, B:const, C:const)$ 
20      2)  $P3(C:const) \mid \sim P4(z2:var) \mid P1(x2:var, y2:var, z2:var)$ 
21
22      Унификация предикатов:
23       $\sim P1(A:const, B:const, C:const)$ 
24       $P1(x2:var, y2:var, z2:var)$ 
25
26      Полученные подстановки:  $\{ 'x2': A:const, 'y2': B:const, 'z2': C:const \}$ 
27      Резольвента:  $P3(C:const) \mid \sim P4(C:const)$ 
28
29      Подстановки:  $\{ 'x2': A:const, 'y2': B:const, 'z2': C:const \}$ 
30      Резольвента:  $P3(C:const) \mid \sim P4(C:const)$ 
31
32      -----
33
34      Пара дизъюнктов:
35      1)  $P3(C:const) \mid \sim P4(C:const)$ 
36      2)  $P2(x1:var, y1:var) \mid P5(z1:var) \mid \sim P6(w1:var)$ 
37
38      Отсутствие унификаций
39
40      -----
41
42      Пара дизъюнктов:
43      1)  $P3(C:const) \mid \sim P4(C:const)$ 
44      2)  $P3(C:const) \mid \sim P4(z2:var) \mid P1(x2:var, y2:var, z2:var)$ 
45
46      Отсутствие унификаций
47
48      -----
49
50      Пара дизъюнктов:
51      1)  $P3(C:const) \mid \sim P4(C:const)$ 
52      2)  $\sim P2(A:const, B:const) \mid P5(x3:var) \mid P6(y3:var)$ 
53
54      Отсутствие унификаций
55
56      -----
57
58      Пара дизъюнктов:
59      1)  $P3(C:const) \mid \sim P4(C:const)$ 
60      2)  $P4(x4:var) \mid \sim P3(x4:var)$ 
61
62      Унификация предикатов:
63       $P3(C:const)$ 
```

Продолжение листинга 1.1

```
64      ~P3(x4:var)
65
66      Полученные подстановки: {'x4': C:const}
67      Резольвента: ~P4(C:const) | P4(C:const)
68
69      Унификация предикатов:
70      ~P4(C:const)
71      P4(C:const)
72
73      Полученные подстановки: {}
74      Резольвента: Пусто
75
76      Подстановки: {'x4': C:const}
77      Резольвента: Пусто
78
79      -----
80
81      Результат резолюции: Пусто
```

2 Текст программы

В листингах 2.1–2.4 представлен код программы.

Листинг 2.1 — Основной модуль программы

```
1  from items import Disjunct
2  from unification import Unification
3  from utils import create_disjunct
4
5
6  def resolve(
7      resolving: Disjunct,
8      knowledge: list[Disjunct],
9      max_counter = 1000
10 ) -> Disjunct:
11     resolving_flag = True
12     while resolving_flag and max_counter > 0:
13         resolving_flag = False
14
15         for rule in knowledge:
16             print("-" * 64, "\n")
17             print(f"Папа дизъюнктов:\n 1) {resolving}\n 2) {rule}\n")
18
19             result = Unification.unificateDisjunct(
20                 left=resolving.copy(),
21                 right=rule.copy(),
22             )
23             if result == None:
24                 print(f"Отсутствие унификаций\n")
25                 continue
26
27             print(f"Подстановки: {result[1]}")
28             print(f"Резольвента: {result[0]}\n")
29
30             resolving = result[0]
31             if len(resolving.args) != 0:
32                 resolving_flag = True
33
34             max_counter -= 1
35             break
36
37     if max_counter == 0:
38         print("Превышено максимальное число итераций\n")
39
40     return resolving
41
```

Продолжение листинга 2.1

```
42
43 def get_knowledge() -> list[Disjunct]:
44     knowledge = [
45         " P2(x1, y1) | P5(z1) | ~P6(w1)",
46         " P3(C)      | ~P4(z2) | P1(x2, y2, z2)",
47         "~P2(A, B)   | P5(x3) | P6(y3)",
48         " P4(x4)     | ~P3(x4)",
49     ]
50     return [create_disjunct(item) for item in knowledge]
51
52
53 def main() -> None:
54     """
55     Процедура резолюции с унификацией
56     """
57     print(
58         "\nБаза знаний:\n", *get_knowledge(), "", sep="\n",
59     )
60     result = resolve(
61         resolving=create_disjunct("~P1(A, B, C)"),
62         knowledge=get_knowledge(),
63     )
64     print("-" * 64, "\n")
65     print(f"Результат резолюции: {result}\n")
66
67
68 if __name__ == "__main__":
69     main()
```

Листинг 2.2 — Модуль с описанием классов термов, атомов и дизъюнк-
ТОВ

```
1 class Term:
2     def __init__(self, name: str, type: str, value=None):
3         self.name = name
4         self.type = type
5         self.value = None
6
7         if type == "const":
8             if value:
9                 self.value = value
10            else:
11                self.value = name
12
13     def __repr__(self):
```


Продолжение листинга 2.2

```
14         return "{:}:{:}".format(self.name, self.type)
15
16     def copy(self):
17         return Term(self.name, self.type, self.value)
18
19
20 class Atom:
21     def __init__(self, name: str, args: list[Term], isPositive=True):
22         self.name = name
23         self.args = args
24         self.isPositive = isPositive
25
26     def __repr__(self):
27         return "{:}({:})".format(
28             '~' if not self.isPositive else '',
29             self.name,
30             ", ".join("{:}".format(term) for term in self.args)
31         )
32
33     def copy(self):
34         return Atom(
35             name=self.name,
36             args=[arg.copy() for arg in self.args],
37             isPositive=self.isPositive
38         )
39
40
41 class Disjunct:
42     def __init__(self, args: list[Atom]):
43         self.args = args
44
45     def __repr__(self):
46         if len(self.args) == 0:
47             return "Пусто"
48
49         return " | ".join("{:}".format(atom) for atom in self.args)
50
51     def copy(self):
52         return Disjunct([arg.copy() for arg in self.args])
```

Листинг 2.3 — Модуль с реализацией класса унификации

```
1 from items import Atom, Disjunct, Term
2
3
```

Продолжение листинга 2.3

```
4 class Unification:
5     @classmethod
6     def unifyDisjunct(
7         cls,
8         left: Disjunct,
9         right: Disjunct
10    ) -> tuple[Disjunct, dict[str, Term]] | None:
11        unificationCount = 0
12        globalSubstitutions = {}
13
14        leftAtoms = left.copy().args
15        rightAtoms = right.copy().args
16
17        while True:
18            localUnificationCount = 0
19
20            for leftAtom in leftAtoms:
21                for rightAtom in rightAtoms:
22                    if leftAtom.name == rightAtom.name:
23                        if leftAtom.isPositive == rightAtom.isPositive:
24                            cls._deleteIdenticalAtom(
25                                leftAtom=leftAtom,
26                                rightAtom=rightAtom,
27                                atomList=rightAtoms,
28                            )
29                        elif cls._tryToUnifyAtoms(
30                            leftAtom=leftAtom,
31                            rightAtom=rightAtom,
32                            leftAtoms=leftAtoms,
33                            rightAtoms=rightAtoms,
34                            globalSubstitutions=globalSubstitutions,
35                        ):
36                            localUnificationCount += 1
37                            break
38
39            if localUnificationCount == 0:
40                break
41
42            unificationCount += localUnificationCount
43
44        if unificationCount == 0:
45            return None
46
47        return Disjunct(leftAtoms + rightAtoms), globalSubstitutions
48
```

Продолжение листинга 2.3

```
49     @classmethod
50     def __tryToUnificateAtoms(
51         cls,
52         leftAtom: Atom,
53         rightAtom: Atom,
54         leftAtoms: list[Atom],
55         rightAtoms: list[Atom],
56         globalSubstitutions: dict[str, Term],
57     ) -> bool:
58         substitutions = cls.__unificateAtoms(
59             left=leftAtom.copy(),
60             right=rightAtom.copy(),
61         )
62         print(
63             f" Унификация предикатов:\n    {leftAtom}\n    {rightAtom}\n\n"\
64             f" Полученные подстановки: {substitutions}"
65         )
66         if substitutions == None:
67             return False
68
69         # удаление контрарных атомов
70         leftAtoms.remove(leftAtom)
71         rightAtoms.remove(rightAtom)
72
73         for sub in substitutions:
74             globalSubstitutions[sub] = substitutions[sub]
75
76         cls.__applySubstitution(leftAtoms, substitutions)
77         cls.__applySubstitution(rightAtoms, substitutions)
78
79         print(f" Резольвента: {Disjunct(leftAtoms + rightAtoms)}\n")
80
81         return True
82
83     @classmethod
84     def __unificateAtoms(
85         cls,
86         left: Atom,
87         right: Atom,
88     ) -> dict[str, Term] | None:
89         if left.name != right.name or len(left.args) != len(right.args):
90             return
91
92         substitutions = {}
93
```

Продолжение листинга 2.3

```

94         for i in range(len(left.args)):
95             leftTerm = left.args[i]
96             rightTerm = right.args[i]
97
98             if leftTerm.type == "const" and rightTerm.type == "const":
99                 if leftTerm.value != rightTerm.value:
100                     return
101
102             elif leftTerm.type == "var" and rightTerm.type == "const":
103                 substitutions[leftTerm.name] = rightTerm
104
105             elif leftTerm.type == "const" and rightTerm.type == "var":
106                 substitutions[rightTerm.name] = leftTerm
107
108             elif leftTerm.type == "var" and rightTerm.type == "var":
109                 if leftTerm.name != rightTerm.name:
110                     substitutions[leftTerm.name] = rightTerm
111
112         return substitutions
113
114     @classmethod
115     def __applySubstitution(
116         cls,
117         atomsList: list[Atom],
118         substitutions: dict[str, Term],
119     ) -> None:
120         for atom in atomsList:
121             for term in atom.args:
122                 if term.name in substitutions:
123                     substitution_term = substitutions[term.name]
124
125                     term.name = substitution_term.name
126                     term.type = substitution_term.type
127
128                     if substitution_term.type == "const":
129                         term.value = substitution_term.value
130
131     @classmethod
132     def __deleteIdenticalAtom(
133         cls,
134         leftAtom: Atom,
135         rightAtom: Atom,
136         atomList: list[Atom],
137     ) -> None:
138         substitutions = cls.__unificateAtoms(
```

Продолжение листинга 2.3

```
139         left=leftAtom.copy(),
140         right=rightAtom.copy()
141     )
142     if substitutions != None and len(substitutions) == 0:
143         atomList.remove(rightAtom)
```

Листинг 2.4 — Модуль с реализацией функции парсера дизъюнкта

```
1  from items import Term, Atom, Disjunct
2
3
4  def create_disjunct(exp: str) -> Disjunct:
5      atoms = [atom.strip() for atom in exp.split("|")]
6      atom_list = []
7
8      for atom in atoms:
9          isPositive = atom[0] != '~'
10         if not isPositive:
11             atom = atom[1:].strip()
12
13         atom_name, atom_args = atom.split('(')
14         atom_name = atom_name.strip()
15         atom_args = atom_args.split(' ')[0].strip()
16         atom_terms = [
17             term.strip() for term in atom_args.split(',')
18         ]
19         terms = [
20             Term(
21                 name=term,
22                 type="const" if term[0].capitalize() == term[0] else "var"
23             ) for term in atom_terms
24         ]
25         atom_list.append(Atom(atom_name, terms, isPositive))
26
27     return Disjunct(atom_list)
```